Flight Delay Prediction via Gradient Boosting and Temporal Deep Learning: Phase 1

***Abstract*—We predict, at the end of the previous day, whether a flight departing New York City will arrive at least 15 minutes late, the U.S. Bureau of Transportation Statistics on-time definition. Using the 2013 New York departures dataset, we build 59 features from the published schedule and from past-only delay outcomes, with explicit safeguards against temporal leakage, and compare L2-regularised Logistic Regression with XGBoost. A strictly chronological protocol uses each period for exactly one decision: January–July for fitting, August for hyperparameter selection, September for probability calibration, October for the decision threshold, and November–December for a single, sealed test evaluation. Every decision was recorded before the results it governs. The two model families perform almost identically on August, and Logistic Regression is selected. On the test set it achieves a PR-AUC of 0.398 against a delay rate of 0.258 and a ROC-AUC of 0.664, consistent with validation. Sigmoid calibration reduces the expected calibration error from 0.204 to 0.077. No threshold reaches the 60% precision target, which reflects the limited information available a day ahead, notably the absence of weather data. Calibration fitted on a calm validation month also under-predicts in the busier test period. We report these limitations transparently and outline extensions with weather data and rolling recalibration.**

***Index Terms*—flight delay prediction, gradient boosting, logistic regression, calibration, temporal leakage**

I. Introduction

Flight delays propagate through crew rotations, aircraft turnarounds and gate schedules. Airline operations teams plan next-day buffers (spare crew, spare aircraft, gate assignments) the evening before, so a calibrated estimate of each flight's delay risk at that point is directly useful. Machine learning for flight delays is an active field [1]. We study the problem as binary classification: will a scheduled flight arrive at least 15 minutes late, the U.S. Bureau of Transportation Statistics on-time definition [2], using only information available at the end of the previous day?

The setting has two pitfalls. First, many informative fields (actual departure time, departure delay, taxi and air time) are known only after departure, and rolling delay statistics can easily include outcomes that were not yet known at prediction time. Either mistake is a form of leakage [3] and produces offline scores that no deployed system could reach. Second, delays cluster in time, so evaluation must respect temporal order [4]. Validating on randomly mixed days, or selecting models on the final test set, overstates performance.

Contributions.

- A leakage-safe feature pipeline of 59 schedule, calendar, carrier, airport, congestion and past-only historical features. Outcome timing is resolved in a common time zone, and tests verify that later records never change earlier features.
- A chronological protocol in which each period serves one purpose (fitting, tuning, calibration, threshold selection, testing). It is backed by a decision log written before the results each decision governs, and by a test set sealed before any analysis and evaluated once.
- A comparison of L2 Logistic Regression and XGBoost with probability calibration, reported with ranking (PR-AUC, ROC-AUC) and calibration (Brier, ECE) metrics.
- A transparent account of what a day-ahead, schedule-only model can and cannot achieve: consistent ranking and good calibration, but no 60%-precision operating point without weather information.

Section II reviews related work, Sections III and IV describe the data and features, Sections V and VI the methodology and protocol, Section VII the results, and Section VIII concludes.

II. Related Work

Flight delay prediction. Machine-learning work on flight delays spans classification and regression, tree ensembles and deep models, and many prediction horizons and delay thresholds [1]. Rebollo and Balakrishnan [5] predict delays 2–24 hours ahead with random forests, using network-wide and link-level delay states. Gui *et al.* [6] combine ADS-B trajectories, weather, schedules and airport information, and report 90.2% accuracy for binary classification with a random forest. Choi *et al.* [7] predict weather-induced delays using weather forecasts. Yi *et al.* [8] stack several classifiers for a single airport and handle class imbalance by oversampling. These studies draw on real-time, weather or trajectory information, and they largely report accuracy or ROC-based metrics without assessing whether predicted probabilities are calibrated.

Delay propagation. Delays spread through airport networks: Pyrgiotis *et al.* [9] model how local congestion propagates and pushes demand into the evening. This motivates our time-of-day, scheduled-congestion and past-only historical delay features.

Methods. We use gradient-boosted trees [10] alongside Logistic Regression. Classifier scores are often poorly calibrated, and Platt scaling and isotonic regression are standard corrections [11], [12]. We report the expected calibration error and reliability diagrams [12] and the Brier score [13]. Because delayed flights are the minority class, precision–recall analysis is more informative than ROC analysis [14], so PR-AUC is our primary metric.

Leakage and time-aware evaluation. Leakage, meaning the use of information that would not be available when predicting, inflates offline results [3]. With temporally dependent data, how a model is validated also matters [4]. We use a chronological hold-out because the model is deployed one day ahead and delay rates shift from month to month.

Gap. We study the stricter day-ahead setting with only the published schedule and past-only outcomes, enforce leakage controls with tests, use each time period for a single decision, and report calibrated probabilities alongside PR-AUC. This quantifies what such a model can and cannot deliver.

III. Dataset and Data Quality

A. Source and target

The dataset, provided for the course, is the Kaggle *Flights* table: all 336,776 flights departing the three New York City airports (EWR, JFK, LGA) in 2013, with 21 columns of scheduled and actual times, carrier, route and distance. The target is $y = 1$ if the arrival delay is at least 15 minutes, the U.S. Bureau of Transportation Statistics on-time convention [2], and $y = 0$ otherwise. Flights without an arrival-delay value (cancelled, or departed with no arrival record) are excluded, so the model estimates the probability of a delay given that the flight operates and arrives. Cancellations and diversions are a separate prediction problem.

B. Chronological split and sealed test set

The data are split by date: training January–August (217,727 labelled flights, 26.4% delayed), validation September–October (55,628, 15.7%) and test November–December (53,991, 25.8%). Before any analysis, the November–December rows were moved out of the development data (the full file's SHA-256 was recorded). Every notebook, feature build and training script reads a January–October file through a loader that rejects later dates. The test rows were read once, in the final evaluation (Section VII).

C. Data quality

On the January–October development data (281,373 rows) there are no duplicate rows, no duplicate flight keys and no error-level violations: all clock times are well formed, and departure and arrival delays agree with actual minus scheduled times once midnight wrap-around is allowed. Missing values are structured rather than random. All actual-time fields are missing together for the 6,997 cancelled flights (2.5%), 1,021 flights departed without an arrival-delay value (0.4%), and the aircraft tail number is missing only for cancelled flights. Because that missingness reveals the outcome, the tail number is not used as a feature. Actual departure and arrival times are post-departure information: departure delay alone reaches a single-feature ROC-AUC of 0.898 against the target, so these fields are excluded from all features.

D. Exploratory analysis on the training period only

All exploratory statistics that informed feature design use January–August only. The delay rate rises steadily through the day, from 10.8% for 05:00 departures to 40.5% at 21:00. Thursday is the worst day (31.0%) and Saturday the best (18.5%). Carriers differ widely (ExpressJet 35.3%, American 20.3%), and the airport effect is not uniform across carriers (e.g. Endeavor 19.1% at EWR vs 29.5% at JFK), which motivates carrier–airport interaction features. The daily delay rate has a lag-1 autocorrelation of 0.356, but lags 2 and 7 are near zero (0.041 and 0.021): yesterday's delays are informative, but persistence beyond that is weak. Monthly delay rates vary between 22.7% and 32.2% within the training period.

IV. Feature Engineering

A. Decision time and leakage policy

Predictions are made at the end of day $D-1$ for every flight on day D, to support next-day operational planning (crew, spare aircraft and gate buffers). A feature may use the published schedule, including all of day D's flights, and the outcomes of earlier flights known before D; nothing realised on day D. Later cut-offs (e.g. two hours before departure) would need same-day real-time data that this dataset cannot validate.

B. Feature set

The model uses 59 features (49 numeric, 10 categorical), fixed before any model was trained:

- **Calendar (10):** scheduled departure hour, day of week, weekend and U.S. federal holiday proximity, with sine/cosine encodings of hour and weekday.
- **Schedule and route (6):** scheduled departure and arrival minutes, scheduled block time, an overnight-arrival flag, distance and log-distance. Block time is computed on a common UTC basis using each airport's IANA time zone, because local clock differences mix duration with time-zone offsets (they imply negative taxi times for 39.2% of flights).
- **Categorical (10):** carrier, origin, destination, route, departure-time bucket and five crosses (carrier with origin, destination, route and bucket; origin with bucket). Levels with fewer than 50 training rows are grouped as *other*.
- **Congestion (4):** scheduled departures from the same origin in the same hour and day, for the same carrier and day, and the hour's share of the day, counted on the full published schedule.
- **Historical (29):** past-only delay rates for carrier, route, origin and destination over 1, 7 and 28 days and all prior days, with counts for the windowed rates, shrunk towards the global past rate with prior strength 20.

Month and day-of-year encodings are excluded: training covers January–August only, so every later month would be outside the training range. Seasonal shifts are left to the past-only history windows.

C. Preventing temporal leakage

A past flight contributes to history only from the New York date on which its outcome was known: its actual arrival time, converted to New York time, plus a 30-minute reporting buffer. On the training period this rule places no label before its arrival date, whereas a simpler departure-plus-air-time estimate would have counted 19 labels a day early. The 30-minute buffer is an assumption about outcome availability (a near-real-time status feed), not a measured value. Learned transformations (rare-level grouping, median imputation, scaling and one-hot encoding) are fitted on training data only. Automated tests check that rebuilding the features with all later records removed leaves every earlier feature value unchanged, that no post-departure or target-derived column enters the feature matrix, and that the feature set stays at the 59 frozen features.

V. Methodology

A. Models

Logistic Regression. The delay probability is $p(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, trained by minimising

$$\tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_i c_{y_i}\, \ell(y_i, p_i), \tag{1}$$

where ℓ is the log-loss, C the inverse regularisation strength and c_y optional class weights. With balanced weights, $c_y = n/(2n_y)$, the delayed class counts more, which shifts predicted probabilities upward. Numeric features are standardised and categorical features one-hot encoded.

XGBoost. Gradient-boosted regression trees [10] add trees f_t that minimise a second-order approximation of the log-loss plus the penalty $\gamma T + \frac{1}{2}\lambda\|\mathbf{w}\|^2$ on leaf count and leaf weights. Tree depth, learning rate, row and column subsampling, and early stopping control complexity. Categorical features use XGBoost's native categorical splits.

B. Metrics

The primary metric is PR-AUC (average precision), $\mathrm{AP} = \sum_n (R_n - R_{n-1}) P_n$, because delayed flights are the minority class [14] and a random ranking scores the delay rate. ROC-AUC is reported alongside. Probability quality is measured by the Brier score [13], $\frac{1}{N}\sum_i (p_i - y_i)^2$, and the expected calibration error over $M = 10$ equal-width bins [12],

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \overline{y}_{B_m} - \overline{p}_{B_m} \right|, \tag{2}$$

with reliability diagrams.

C. Calibration and decision threshold

Sigmoid (Platt) calibration fits $p_{\text{cal}} = \sigma(as + b)$ on held-out scores s of the frozen model [11]. Isotonic regression is reported for comparison only. Because calibration is monotone, it fixes the probability scale without changing the ranking. The operating threshold is the one with the highest recall subject to precision of at least 0.60. Planners act on every flagged flight, so false alarms carry a cost. The 60% level is a stated assumption, as no cost data are available.

D. One job per period

Each time period is used for exactly one decision, so no period scores a choice it helped make: January–July to fit candidate models, August to tune hyperparameters (and for XGBoost early stopping), January–August to retrain the chosen model, September to fit calibration, October to choose the threshold, and November–December for a single test evaluation.

VI. Experimental Setup

A. Pre-registered protocol

Every decision was written into a dated decision log and committed *before* the results it governs existed. This covers the feature set, the hyperparameter grid, the selection rule, the calibration method, the threshold rule and the test procedure. An earlier iteration of this project evaluated on the November–December holdout. All models and features reported here were re-developed and selected without reference to those results.

B. Hyperparameter search and model selection

Both model families were trained on January–July and scored on August:

- **Logistic Regression** (L2, up to 2,000 iterations): $C \in \{0.01, 0.1, 1, 10\}$ $\times$ class weight $\in \{\text{none}, \text{balanced}\}$, 8 runs.
- **XGBoost** (up to 1,000 trees, early stopping after 50 rounds without August improvement, subsample and column sample 0.8, `scale_pos_weight` fixed at 1): depth $\in \{3, 5\}$ $\times$ learning rate $\in \{0.03, 0.1\}$, 4 runs.

The rule fixed in advance selects the run with the highest August PR-AUC, with ties going to the simpler model. The chosen configuration is then retrained once on January–August, with preprocessing refitted on the same period; for XGBoost the tree count would be fixed at the August-selected value with early stopping off. August scores are therefore tuning scores, not performance estimates. The encoder for categorical levels and missing values is refitted on January–July for the candidate runs, so August information does not reach them.

C. Calibration, threshold and test procedure

Sigmoid calibration is fitted on September (27,010 flights) for the retrained model, with the model itself frozen. The threshold is chosen on October (28,618 flights) by the rule in Section V. If no threshold reaches 0.60 precision, the result is recorded as infeasible and the target is not lowered. The calibrated model and operating point were then committed and tagged. The single test evaluation checks that the frozen files are byte-identical to the tagged versions and refuses to run a second time once results exist. It builds features over the whole year, so that test flights receive their correct past-only history, and scores only November–December.

D. Reproducibility

Random seed 42 is used throughout. Training used Python 3.11.17, scikit-learn 1.9.1 and XGBoost 3.2.0. Every run's hyperparameters and August scores are saved with the trained models, and the scripts that produced each result refuse to overwrite their outputs. Logistic Regression results reproduce exactly from the logged settings. XGBoost early stopping is sensitive to the library version: the best XGBoost run reproduced at 0.393 rather than 0.396 under XGBoost 3.4.1, which does not change the selected model.

VII. Results and Discussion

A. Model selection on August

Table I summarises the 12 candidate runs. All Logistic Regression settings scored within 0.001 PR-AUC of each other, and the best XGBoost run (depth 5, learning rate 0.1, 16 trees) was 0.005 lower. Logistic Regression with $C = 0.01$ and balanced class weights was selected. That two very different model families perform almost identically suggests the limit is the information in the features rather than model capacity. The August delay rate is 24.6%, so the selected model ranks delayed flights about 1.6 times better than random (Fig. 1).

TABLE I
Candidate models, trained January–July, scored on August

Model	Setting	PR-AUC	ROC-AUC
LR (selected)	C=0.01, balanced	**0.4007**	0.6756
LR (range, 8 runs)	—	0.3998–0.4007	0.6733–0.6756
XGBoost (best)	depth 5, lr 0.1	0.3960	0.6770
XGBoost (range, 4 runs)	—	0.3928–0.3960	0.6717–0.6770

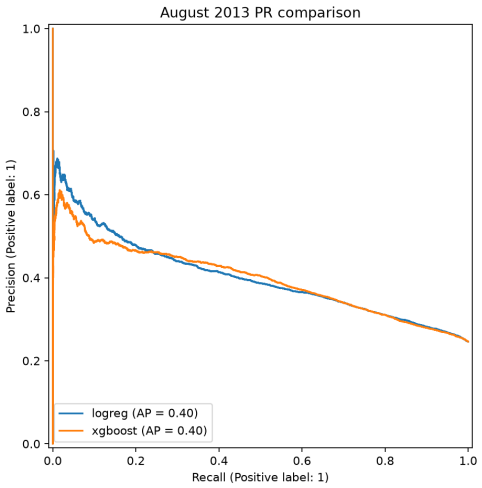


Fig. 1. Precision–recall curves on August for the best Logistic Regression and XGBoost candidates.

Feature importance. For the tuned XGBoost model, the largest gain comes from scheduled departure time (`dep_hour_sin`, `sched_dep_hour_frac`), the previous day's carrier delay rate, the carrier–departure-time cross and scheduled origin volume. Permutation importance on August ranks the carrier–departure-time cross, the carrier–route cross, origin daily volume and departure time highest. Time of day, carrier and recent delays carry most of the signal, matching the exploratory analysis. Permutation estimates use 2,000 August flights and two repeats, so they are indicative only.

B. Calibration and threshold (September and October)

The selected model's raw probabilities are far too high (Table II), as expected from balanced class weights. Sigmoid calibration fitted on September reduces October ECE from 0.227 to 0.027 and Brier score from 0.192 to 0.136, without changing PR-AUC or ROC-AUC. Isotonic calibration was slightly worse on October and was not used.

TABLE II
Calibration (fitted on September; October is out-of-sample)

Period	Probabilities	Brier	ECE	Mean p / delay rate
Sep	raw	0.179	0.240	0.377 / 0.137
Sep	sigmoid	0.113	0.010	0.137 / 0.137
Oct	raw	0.192	0.227	0.403 / 0.176
Oct	sigmoid	0.136	0.027	0.148 / 0.176
Oct	isotonic (comparison)	0.137	0.029	0.147 / 0.176

No threshold reached 60% precision on October, so, as planned, no threshold was set and the rescheduling rule was not applied. As a diagnostic only, the highest October precision with at least 20 delayed flights flagged was 51.3% (158 flights flagged, recall 1.6%), and no October flight received a calibrated probability above 0.525.

C. Test results (November–December, evaluated once)

Table III reports the single evaluation on the sealed test set. PR-AUC is 0.398 against a delay rate of 0.258, about 1.5 times a random ranking, and ROC-AUC is 0.664. These are close to the validation values (ROC-AUC 0.67–0.69), so the model generalises to the later period without collapse. Calibration reduces ECE from 0.204 to 0.077 and Brier score from 0.226 to 0.185. The calibrated probabilities are, however, too low on average (0.182 predicted vs 0.258 observed; Fig. 2), because the calibration was fitted on September, the calmest month (13.7% delayed). Threshold-based metrics are not available, because no operating point met the precision target on October.

TABLE III
Single test evaluation, November–December (53,991 flights)

Probabilities	PR-AUC	ROC-AUC	Brier	ECE
Raw	0.398	0.664	0.226	0.204
Calibrated (final)	0.398	0.664	**0.185**	**0.077**

D. Discussion

At the end of the previous day, the dataset offers only the schedule and past outcomes. The largest drivers of delay, such as weather and same-day propagation, are unavailable. The model therefore separates riskier from safer flights consistently, but it is rarely confident about an individual flight,

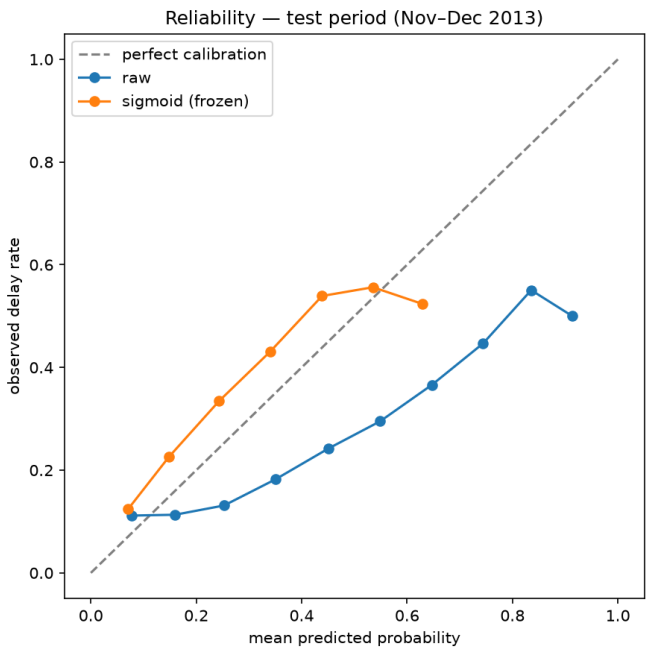


Fig. 2. Reliability diagram on the test period. Raw probabilities are too high; calibrated probabilities lie above the diagonal, i.e. too low, because calibration was fitted on a calmer month.

which is why a 60% precision operating point does not exist. The validation period was also calmer than both training and test, which affects anything fitted on it, most visibly calibration.

VIII. Conclusion and Future Work

This work predicts, at the end of the previous day, whether a New York departure will arrive at least 15 minutes late. It uses 59 leakage-safe features built from the published schedule and past-only outcomes, Logistic Regression and XGBoost, and a strictly chronological protocol in which each period is used for one decision. The test period was sealed before any analysis and evaluated once. On it, the selected L2 Logistic Regression achieves PR-AUC 0.398 (delay rate 0.258) and ROC-AUC 0.664. Sigmoid calibration reduces the expected calibration error from 0.204 to 0.077.

Limitations.

- The provided dataset has no weather or same-day operational information, which bounds how confident any flight-level prediction can be.
- The validation months were calmer than training and test, so the calibration fitted on September under-predicts on the test period.
- The 60% precision target is an assumption, not a measured cost trade-off, and was not reachable. No operating threshold or rescheduling rule could therefore be evaluated.
- History features assume flight outcomes are available within 30 minutes of arrival.
- One year of departures from three airports limits generalisation to other years and airports.

Future work. Adding weather forecasts available at $D-1$ is the most promising extension. Recalibrating on recent data, such as a rolling recalibration window, would address the seasonal shift in delay rates. With real cost estimates for false alarms and missed delays, the operating point and the offline rescheduling rule could be set by expected cost. Phase 2 will explore a compact neural model with learned embeddings for airline, origin and destination.

References

[1] L. Carvalho, A. Sternberg, L. Maia Gonçalves, A. B. Cruz, J. A. Soares, D. Brandão, D. Carvalho, and E. Ogasawara, "On the relevance of data science for flight delay research: a systematic review," *Transport Reviews*, vol. 41, no. 4, pp. 499–528, 2021.

[2] Bureau of Transportation Statistics, "Airline on-time performance data (data profile)," TranStats, U.S. Department of Transportation, accessed 25 October 2026. [Online]. Available: https://www.transtats.bts.gov/DatabaseInfo.asp?QO_VQ=EFD

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[4] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[5] J. J. Rebollo and H. Balakrishnan, "Characterization and prediction of air traffic delays," *Transportation Research Part C: Emerging Technologies*, vol. 44, pp. 231–241, 2014.

[6] G. Gui, F. Liu, J. Sun, J. Yang, Z. Zhou, and D. Zhao, "Flight delay prediction based on aviation big data and machine learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 1, pp. 140–150, 2020.

[7] S. Choi, Y. J. Kim, S. Briceño, and D. Mavris, "Prediction of weather-induced airline delays based on machine learning algorithms," in *2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, 2016, pp. 1–6.

[8] J. Yi, H. Zhang, H. Liu, G. Zhong, and G. Li, "Flight delay classification prediction based on stacking algorithm," *Journal of Advanced Transportation*, vol. 2021, pp. 1–10, 2021.

[9] N. Pyrgiotis, K. M. Malone, and A. Odoni, "Modelling delay propagation within an airport network," *Transportation Research Part C: Emerging Technologies*, vol. 27, pp. 60–75, 2013.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[11] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, 2005, pp. 625–632.

[12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[13] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[14] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.